

Nonlinear Factor Decomposition via Kolmogorov-Arnold Networks:

A Spectral Approach to Asset Return Analysis

David Breazu

Faculty of Mathematics and Computer Science, University of Bucharest

Bucharest, Romania

March 2026

Abstract

KAN-PCA is an autoencoder that uses a KAN as encoder and a linear map as decoder. It generalizes classical PCA by replacing linear projections with learned B-spline functions on each edge. The motivation is to capture more variance than classical PCA, since the linear assumption of PCA can misrepresent the dependence structure during market stress, when correlations between assets change sharply.

We prove that if the spline activations are forced to be affine, KAN-PCA yields exactly the same result as classical PCA, which establishes PCA as a strict special case of the model. Experiments on 20 S&P 500 stocks show that KAN-PCA captures slightly more in-sample variance than classical PCA at equal factor count ($R^2 = 65.56\%$ vs 64.71% with 3 factors on the training period), while out-of-sample the two methods are close, with classical PCA marginally ahead (57.32% vs 56.73%). All results use a training-only benchmark that removes the data leakage present in our initial experiments.

Contents

1	Introduction	4
2	Related Work	4
3	Mathematical Framework	6
3.1	Classical PCA	6
3.2	B-Spline Functions	6
3.3	Kolmogorov-Arnold Networks	7
3.4	KAN-PCA	7
4	Theoretical Result	8
5	Experiments	10
5.1	Data and Setup	10
5.2	Models	10
5.3	Results	10
5.4	Ablation Study	11
5.5	Factor Analysis	12
5.6	Edge Function Analysis	13
6	Conclusion	13

Introduction

A factor model is a useful tool for understanding a portfolio of assets. Instead of analyzing every stock individually, a factor model identifies a small number of common forces that drive all assets simultaneously. These forces are called factors [4]. For a hedge fund, this means that instead of tracking hundreds of individual positions, one can describe the entire portfolio through a handful of underlying risk sources, which reduces a high-dimensional problem to an interpretable one.

Classical PCA is the standard method for extracting these factors from return data. However, PCA cannot capture the nonlinearities present in financial markets. During market stress, asset correlations rise sharply, so a fixed linear factor structure estimated over a full sample can misrepresent the dependence structure in such periods. PCA treats a crisis as a linear amplification of normal market dynamics rather than as a qualitatively different regime.

We therefore propose KAN-PCA: an autoencoder that uses a KAN encoder [9] combined with a linear decoder. The KAN encoder replaces linear projections with learned spline functions, while the linear decoder keeps the factors directly interpretable. When the spline activations are forced to be affine, KAN-PCA recovers classical PCA exactly. In the presence of genuine nonlinearity, it captures additional structure that PCA cannot represent by construction.

Our main contributions are:

1. A formal proof that forcing the spline activations to be affine yields exactly the same result as classical PCA, establishing PCA as a special case of the model.
2. Experimental evidence that KAN-PCA captures slightly more in-sample variance than classical PCA with the same number of factors (65.56% vs 64.71% on the training period).
3. An ablation study that identifies and corrects a data leakage issue in the original training procedure. Under a training-only benchmark, classical PCA and KAN-PCA are close out-of-sample, with PCA marginally ahead (57.32% vs 56.73%).
4. Visualization of the learned edge functions, providing direct evidence of nonlinear factor structure in S&P 500 returns.

Related Work

Nonlinear generalizations of PCA. Classical PCA is the optimal linear method for second-order dimensionality reduction, but its linearity is a known limitation when the data lies on a nonlinear manifold. Two classical generalizations address this. Kernel PCA

[12] maps the data into a high-dimensional feature space through a fixed kernel and performs linear PCA there, so the nonlinearity is fixed in advance by the choice of kernel rather than learned from data. Autoencoders provide the second route: a neural encoder and decoder trained on a reconstruction objective learn a nonlinear low-dimensional representation. The connection between the two is classical. A linear autoencoder with squared loss recovers the PCA subspace exactly [1], and deep autoencoders were introduced as a practical nonlinear PCA by Hinton and Salakhutdinov [7]. KAN-PCA belongs to the autoencoder family, but it differs from a generic nonlinear autoencoder in that its nonlinearity is carried by learnable univariate B-spline functions on each edge, which keeps each asset’s contribution individually inspectable rather than entangled in a dense weight matrix.

Autoencoders in asset pricing. In finance, autoencoders have been used to estimate latent factor structures from returns and characteristics. Gu et al. [6] introduced conditional autoencoder asset pricing models, in which factor loadings are modeled as nonlinear functions of firm characteristics, establishing the autoencoder as a standard tool for latent factor estimation in empirical asset pricing. Our work is narrower and more classical in spirit: we extract factors directly from the return covariance structure, without conditioning on characteristics, which keeps the comparison with classical PCA exact and controlled.

KAN-based models. Kolmogorov-Arnold Networks [9] have recently been combined with autoencoders for representation learning, for example the Kolmogorov-Arnold Auto-Encoder of Yu et al. [14] and the KAN autoencoders of Moradi et al. [11], both of which report improved reconstruction over MLP autoencoders on benchmark data. Most directly related to the present work, Wang and Singh [13] propose a KAN-based autoencoder for conditional asset pricing, modeling factor exposures as nonlinear functions of asset characteristics. We differ from Wang and Singh [13] in two respects. First, our object of study is the covariance structure of returns rather than characteristic-conditioned exposures, which places our model in a direct, like-for-like comparison with classical PCA. Second, and central to this paper, we establish a theoretical characterization absent from prior KAN-autoencoder work: we prove that constraining the spline activations to be affine makes KAN-PCA coincide exactly with classical PCA, so that PCA is recovered as a strict special case within a larger function class. To our knowledge this exact-recovery result has not previously been stated for KAN autoencoders.

Mathematical Framework

Classical PCA

Let $\{x_t\}_{t=1}^T \subset \mathbb{R}^N$ be a sequence of N -dimensional standardized return vectors with zero mean. The empirical covariance matrix is:

$$\Sigma := \frac{1}{T} \sum_{t=1}^T x_t x_t^\top \in \mathbb{R}^{N \times N}, \quad (\Sigma_{ij} = \text{empirical covariance between asset } i \text{ and asset } j)$$

By the spectral theorem, since Σ is symmetric positive semidefinite, it admits the decomposition:

$$\Sigma = U \Lambda U^\top, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_N), \quad \lambda_1 \geq \dots \geq \lambda_N \geq 0$$

where $U = [u_1 \mid \dots \mid u_N]$ satisfies $U^\top U = I$ (the columns are orthonormal: each eigenvector has unit length and is perpendicular to all others). The k -factor PCA projection uses the top- k eigenvectors $U_k := [u_1 \mid \dots \mid u_k]$:

$$z_t = U_k^\top x_t \in \mathbb{R}^k, \quad \hat{x}_t = U_k z_t \in \mathbb{R}^N$$

The reconstruction loss equals $\sum_{i=k+1}^N \lambda_i$, the minimum achievable among all rank- k linear approximations. On the full sample with $N = 20$ stocks and $k = 3$, the leading eigenvalues account for $R^2 = 62.99\%$ of the total variance, with:

$$\lambda_1 \text{ explains } 48.00\%, \quad \lambda_2 \text{ explains } 8.96\%, \quad \lambda_3 \text{ explains } 6.03\%$$

(The experiments in Section 5 instead fit PCA on the training period only, for a fair comparison against the learned models, which gives a slightly different in-sample figure.)

B-Spline Functions

Definition 1 (B-Spline Space). *Let $G = \{t_0 < t_1 < \dots < t_m\} \subset \mathbb{R}$ be a knot vector and $k \geq 1$. The B-spline space of degree k over G is:*

$$\mathcal{S}_k(G) := \left\{ \varphi \in C^{k-1}(\mathbb{R}) \mid \varphi|_{[t_i, t_{i+1}]} \in \mathcal{P}_k, \forall i \right\}$$

where $\mathcal{P}_k$ denotes the space of polynomials of degree at most k [2].

We use k to denote the polynomial degree, so a degree- k spline has C^{k-1} continuity at simple interior knots (equivalently, order $k+1$ in de Boor's convention). B-splines are piecewise polynomials that are smooth at their knots. A cubic B-spline ($k=3$) is therefore C^2 -continuous, which is sufficient smoothness for our purposes. The basis functions $\{B_{i,k}\}$ are computed via the Cox-de Boor recursion and satisfy the partition of unity property

$$\sum_i B_{i,k}(x) = 1 \text{ [2].}$$

Kolmogorov-Arnold Networks

The Kolmogorov-Arnold representation theorem [8] states that any continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ can be written as:

$$f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \varphi_{q,p}(x_p) \right)$$

where $\Phi_q, \varphi_{q,p} : \mathbb{R} \rightarrow \mathbb{R}$ are continuous univariate functions.

Definition 2 (KAN Layer). *A KAN layer $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is defined componentwise by:*

$$(\Phi(x))_j := \sum_{i=1}^n \varphi_{ji}(x_i), \quad j = 1, \dots, m$$

where each $\varphi_{ji} \in \mathcal{S}_k(G)$ is a learnable univariate B-spline function [9]. The full activation combines a SiLU [3] basis term with the spline:

$$\varphi_{ji}(x) = w_b \cdot \text{SiLU}(x) + w_s \cdot \sum_l c_l B_{l,k}(x)$$

with learnable parameters $w_b, w_s \in \mathbb{R}$ and spline coefficients $c_l \in \mathbb{R}$.

The key distinction from MLPs is that the activations reside on edges. Each edge carries its own learned function rather than a scalar weight, and the nodes merely sum their inputs.

KAN-PCA

Definition 3 (KAN-PCA). *KAN-PCA is a KAN autoencoder consisting of:*

- A KAN encoder $\mathcal{E} : \mathbb{R}^N \rightarrow \mathbb{R}^k$, a composition of KAN layers;
- A linear decoder $\mathcal{D} : \mathbb{R}^k \rightarrow \mathbb{R}^N$, defined by $\mathcal{D}(z) = Wz$ where $W \in \mathbb{R}^{N \times k}$ is the decoding matrix: row i of W specifies how asset i is reconstructed from the k factors;

trained to minimize the reconstruction loss:

$$\mathcal{L}(\mathcal{E}, \mathcal{D}) := \frac{1}{T} \sum_{t=1}^T \|x_t - \mathcal{D}(\mathcal{E}(x_t))\|^2$$

The k -dimensional representations $z_t = \mathcal{E}(x_t) \in \mathbb{R}^k$ are the nonlinear factors: the compressed description of the market on day t .

Unlike classical PCA, each asset’s contribution to each factor is mediated by a learned spline function φ_{ji} rather than a fixed scalar weight. This lets the model capture asymmetric and regime-dependent relationships between assets and factors.

Theoretical Result

Lemma 1 (Affine Splines are Linear Maps). *A spline function $\varphi \in \mathcal{S}_k(G)$ whose second derivative vanishes identically is affine, $\varphi(x) = wx + b$ for some $w, b \in \mathbb{R}$. For zero-mean standardized inputs the intercept b does not affect the recovered factor subspace, so it may be set to $b = 0$ without loss of generality, giving $\varphi(x) = wx$.*

Proof. A degree-1 polynomial is automatically an element of any spline space $\mathcal{S}_k(G)$ with $k \geq 1$. If the second derivative is zero everywhere, the function does not curve at any point, so it is affine: $\varphi(x) = wx + b$. When the inputs x_t are centered to zero mean, a constant shift b applied on each edge contributes only a fixed offset to the layer output and is absorbed without changing the covariance structure or the recovered subspace; we therefore take $b = 0$. $\square$

Lemma 2 (Linear KAN Layer is Matrix Multiplication). *If all spline functions $\varphi_{ji} \in \mathcal{S}_k(G)$ satisfy $\varphi_{ji}'' \equiv 0$ for all i, j , then the KAN layer $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ reduces to:*

$$\Phi(x) = Wx, \quad W_{ji} := w_{ji} \in \mathbb{R}$$

Proof. By Lemma 1, $\varphi_{ji}(x_i) = w_{ji}x_i$ for each edge (j, i) . Therefore:

$$(\Phi(x))_j = \sum_{i=1}^n \varphi_{ji}(x_i) = \sum_{i=1}^n w_{ji}x_i = (Wx)_j$$

where $W = (w_{ji}) \in \mathbb{R}^{m \times n}$. The full layer computes $x \mapsto Wx$.

Concrete example (3 stocks, 3 nodes):

$$\begin{pmatrix} \text{node}_1 \\ \text{node}_2 \\ \text{node}_3 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{pmatrix} \begin{pmatrix} x_{\text{AAPL}} \\ x_{\text{MSFT}} \\ x_{\text{GOOG}} \end{pmatrix} = Wx$$

$\square$

Lemma 3 (Optimal Linear Autoencoder = PCA [1]). *Let $\Sigma = U\Lambda U^\top$ be the spectral decomposition of the empirical covariance matrix. The linear autoencoder:*

$$\min_{A \in \mathbb{R}^{k \times N}, W \in \mathbb{R}^{N \times k}} \frac{1}{T} \sum_{t=1}^T \|x_t - WAx_t\|^2$$

achieves its global minimum at $A = U_k^\top$, $W = U_k$, where U_k are the top- k eigenvectors of Σ . The minimum value is $\sum_{i=k+1}^N \lambda_i$.

Proof. Expanding the loss using $\Sigma = \frac{1}{T} \sum_t x_t x_t^\top$:

$$\mathcal{L}(A, W) = \text{tr}(\Sigma - \Sigma A^\top W^\top - W A \Sigma + W A \Sigma A^\top W^\top)$$

Setting $\frac{\partial \mathcal{L}}{\partial W} = 0$ gives $W = \Sigma A^\top (A \Sigma A^\top)^{-1}$. Substituting back:

$$\mathcal{L}(A) = \text{tr}(\Sigma) - \text{tr}(A \Sigma A^\top (A A^\top)^{-1})$$

By the Ky Fan maximum principle [5], the maximum of $\text{tr}(A \Sigma A^\top (A A^\top)^{-1})$ is attained when the rows of A span the top- k eigenspace of Σ , giving $A = U_k^\top$ and $W = U_k$. The minimum loss is $\mathcal{L}^* = \sum_{i=k+1}^N \lambda_i$. $\square$

Theorem 4 (KAN-PCA Reduces to PCA in the Linear Limit). *Let KAN-PCA be a KAN autoencoder $\mathcal{E}$ with linear decoder $\mathcal{D}$, trained to minimize the reconstruction loss. If all spline activations φ_{ji} are constrained to satisfy $\varphi''_{ji} \equiv 0$ everywhere, then the optimal encoder and decoder satisfy:*

$$\mathcal{E}^*(x) = U_k^\top x \quad \mathcal{D}^*(z) = U_k z$$

where $U_k^\top x$ is a standard matrix multiplication, the linear limit of the KAN encoder by Lemmas 1 and 2. Consequently:

$$\mathcal{L}_{\text{KAN}}^* \leq \mathcal{L}_{\text{PCA}}^* = \sum_{i=k+1}^N \lambda_i$$

with equality if and only if the data admits a purely linear factor structure.

Proof. By Lemmas 1 and 2, the KAN encoder reduces to $\mathcal{E}_{\text{lin}}(x) = Ax$. The loss becomes the linear autoencoder objective of Lemma 3, giving $A^* = U_k^\top$ and $W^* = U_k$. $\square$

Corollary 5. *Classical PCA is a special case of KAN-PCA:*

$$\text{KAN-PCA} \Big|_{\varphi''_{ji} \equiv 0} = \text{PCA}$$

Writing $\mathcal{L}_{\text{aff}}$ for the affine functions, the inclusion $\mathcal{L}_{\text{aff}} \subsetneq \mathcal{S}_k(G)$ for $k \geq 2$ is strict, so KAN-PCA optimizes over a strictly larger function class than PCA.

Experiments

Data and Setup

We use daily log-returns for 20 S&P 500 large-cap constituents from January 2015 to January 2024, comprising 2,263 trading days. The tickers are AAPL, MSFT, GOOGL, JPM, JNJ, XOM, BAC, PG, KO, WMT, CVX, GS, HD, MCD, CAT, MMM, IBM, GE, BA, and DIS. Returns are standardized to zero mean and unit variance per stock.

The dataset is split 70/10/20: a training set (1,584 days, 2015–2020), a validation set (226 days, 2021), and a test set (453 days, 2022–2024). All hyperparameter choices are made on the training and validation data only.

Models

We benchmark KAN-PCA against classical PCA, all using $k = 3$ factors. Classical PCA is the standard spectral decomposition of Σ , fitted on the training period only to give a fair, leakage-free benchmark. The KAN-PCA encoder uses the architecture $[20, 10, 3, 20]$: 20 stock inputs, an intermediate KAN layer of 10 neurons, compression to 3 factors, and a linear decoder back to 20 outputs.

We arrived at the final configuration through several iterations. Initial variants (a simpler $[20, 3, 20]$ encoder and a deeper $[20, 10, 3, 20]$ encoder with grid $G = 10$) were trained without a proper validation split, which, as we explain in the ablation study, led to data leakage and optimistic out-of-sample numbers. The final model, KAN-PCA Improved, corrects this with a proper 70/10/20 split, early stopping on the validation loss, progressive grid extension ($3 \rightarrow 5 \rightarrow 10$), and mild regularization (spline penalty λ decreasing across stages from 10^{-3} to 10^{-4} , entropy penalty 0.1). All numbers reported below are from this corrected procedure.

Results

Table 1: Classical PCA versus KAN-PCA on 20 S&P 500 stocks ($k = 3$ factors, 70/10/20 split). Both models are fitted on the training period only.

Model	R^2 Train	R^2 Out-of-Sample
Classical PCA (train-fit)	64.71%	57.32%
KAN-PCA Improved	65.56%	56.73%

R^2 Train = *in-sample explained variance on the training period*. R^2 Out-of-Sample = *explained variance on the held-out test period*.

On the training period, KAN-PCA captures slightly more variance than classical PCA (65.56% vs 64.71%), which indicates that some nonlinear structure exists in the returns that a purely linear projection cannot represent. Out-of-sample the two methods are close, with classical PCA marginally ahead by 0.59 percentage points (57.32% vs 56.73%). In other words, the additional flexibility of the spline activations helps in-sample but does not translate into an out-of-sample advantage on this dataset and universe size. This is consistent with the theoretical result of Section 3: when the underlying factor structure is close to linear, the larger function class of KAN-PCA has little genuine nonlinearity to exploit, and it reduces to behaviour very near that of PCA.

Ablation Study

The first versions of the model were trained and evaluated without a proper separation between training and test data. This created a data leakage issue: the pykan training routine uses the evaluation set for internal grid updates, so the model indirectly observed test-set statistics while learning. As a result, the early out-of-sample numbers were optimistic and not reproducible under a clean protocol.

The corrected model uses a proper 70/10/20 train/validation/test split, early stopping on the validation loss, progressive grid extension ($3 \rightarrow 5 \rightarrow 10$), and mild regularization. Under this protocol the model trains stably and the comparison against PCA is fair: KAN-PCA is 0.59pp below classical PCA out-of-sample (56.73% vs 57.32%) while being slightly ahead in-sample. The lesson of the ablation is methodological. Most of the apparent out-of-sample strength in the first experiments was an artifact of leakage rather than a real property of the model, and a careful protocol is essential when evaluating flexible nonlinear models on financial time series.

Factor Analysis

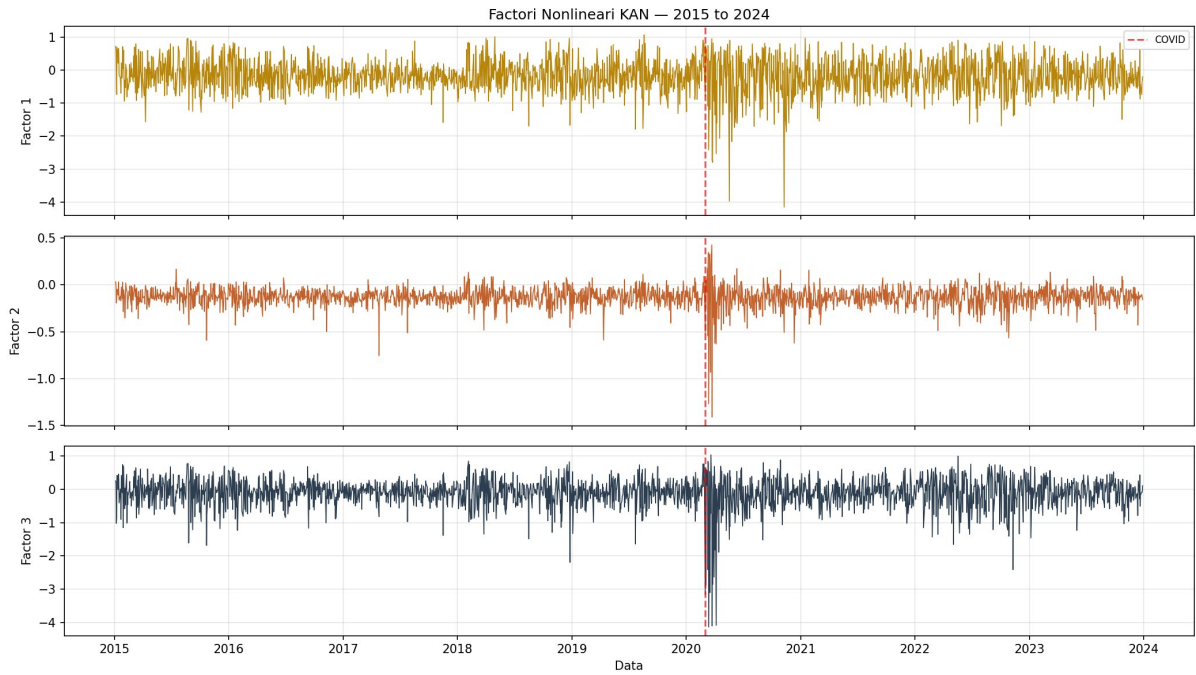


Figure 1: KAN nonlinear factors (2015–2024). The red dashed line marks the COVID-19 crash of March 2020. All three factors show sharp movements at this date, in contrast to the linear response of classical PCA.

All three KAN factors react sharply at the COVID-19 crash. On the standardized scale, each factor drops well below its normal range during March 2020: Factor 1 and Factor 3 fall toward -4 , and Factor 2 toward -1.5 , in each case several times their typical daily amplitude. The qualitative point is that the model registers the crash as a large, abrupt deviation in all three coordinates simultaneously, whereas classical PCA spreads the same shock linearly across its components. The magnitudes themselves should be read as relative excursions rather than calibrated quantities, since the factors are only identified up to scale.

Edge Function Analysis

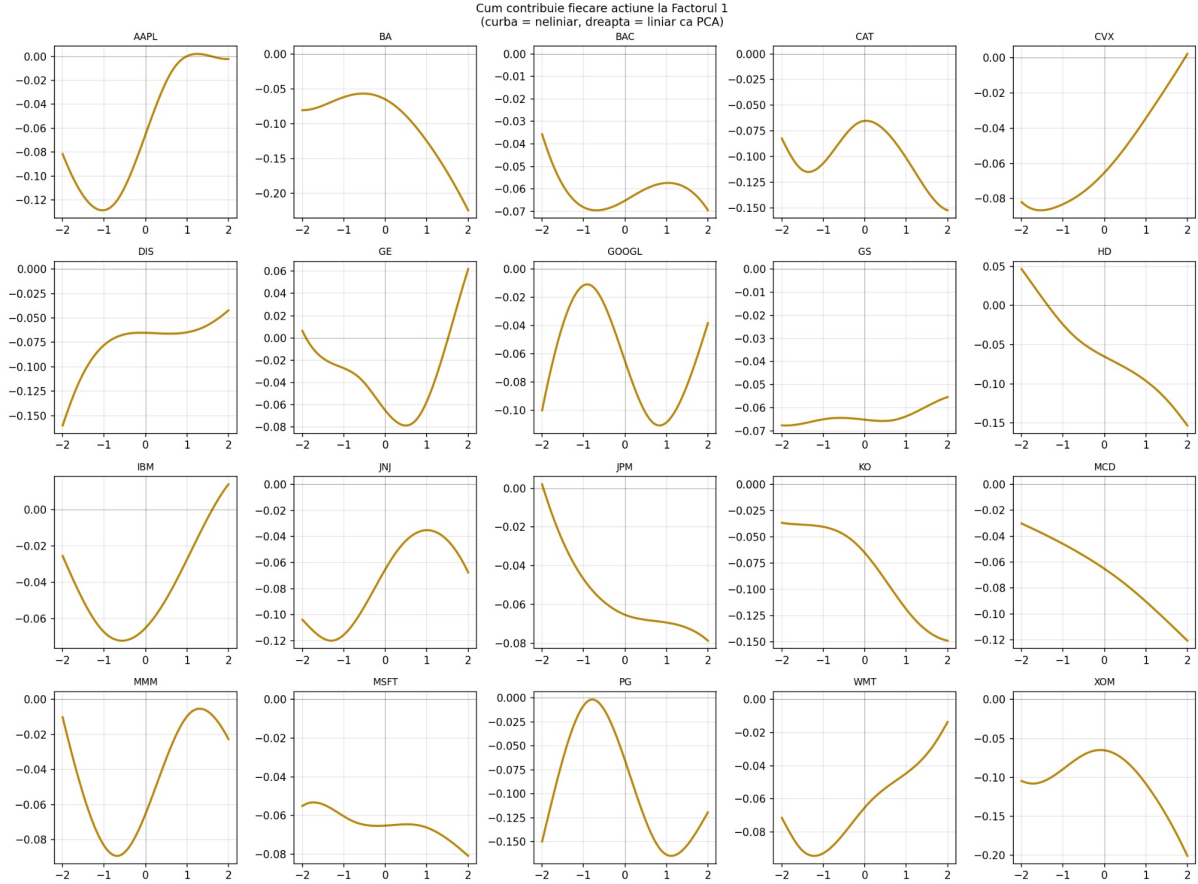


Figure 2: Learned edge functions $\varphi(x)$ for all 20 stocks (contribution to Factor 1). If the relationships were linear, as PCA assumes, all curves would be straight lines. They are not, which gives direct visual evidence of nonlinear factor structure.

The learned edge functions $\varphi_{ji}(x)$ are visibly nonlinear across the 20 stocks; a strictly linear PCA loading would appear as a straight line in every panel. Several patterns stand out. DIS shows a sharp localized peak near zero, so small returns contribute very differently from a linear extrapolation. HD is nearly flat for negative returns and then rises steeply for positive ones, an asymmetric response. PG is bimodal, with two distinct local minima, consistent with more than one operating regime. Other names such as KO, MCD, and XOM are closer to monotone but still curved, indicating mild rather than strong nonlinearity. Taken together, the panels show that the nonlinearity captured by the model is real but uneven across assets.

Conclusion

In this paper we studied KAN-PCA, a factor model that replaces the linear projections of classical PCA with learned spline functions on each edge while keeping a linear, in-

interpretable decoder. Our main result is theoretical: when the spline activations are constrained to be affine, KAN-PCA coincides exactly with classical PCA, so PCA is a strict special case within a larger function class. Empirically, on 20 S&P 500 stocks the model captures slightly more variance than PCA in-sample (65.56% vs 64.71%), while out-of-sample the two are close, with PCA marginally ahead (57.32% vs 56.73%). The learned edge functions are visibly nonlinear, which gives direct evidence that some nonlinear structure is present even though it does not translate into an out-of-sample gain at this universe size.

We also reported a methodological lesson. Our first experiments suffered from data leakage through the grid-update routine, which produced optimistic out-of-sample numbers; a proper validation split removes this effect and yields the fair comparison above.

A natural next step is to apply the same construction to a larger universe of assets, on the order of 200 stocks, with a deeper architecture and a longer training procedure. Our preliminary attempts at that scale did not train stably, and we leave a careful treatment, including the sparsification and grid-extension techniques of KAN 2.0 [10], to future work. Whether the larger function class of KAN-PCA yields a genuine out-of-sample advantage on returns data, rather than only an in-sample one, remains an open empirical question.

References

- [1] Baldi, P. and Hornik, K. (1989). Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks*, 2(1):53–58.
- [2] de Boor, C. (2001). *A Practical Guide to Splines* (Revised Edition). Applied Mathematical Sciences, vol. 27. Springer-Verlag, New York.
- [3] Elfving, S., Uchibe, E., and Doya, K. (2018). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11.
- [4] Fama, E. F. and French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1):3–56.
- [5] Fan, K. (1949). On a theorem of Weyl concerning eigenvalues of linear transformations. I. *Proceedings of the National Academy of Sciences USA*, 35(11):652–655.
- [6] Gu, S., Kelly, B., and Xiu, D. (2021). Autoencoder asset pricing models. *Journal of Econometrics*, 222(1):429–450.
- [7] Hinton, G. E. and Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507.
- [8] Kolmogorov, A. N. (1957). On the representation of continuous functions of many

- variables by superposition of continuous functions of one variable and addition. *Doklady Akademii Nauk SSSR*, 114(5):953–956. (In Russian.)
- [9] Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljačić, M., Hou, T. Y., and Tegmark, M. (2024a). KAN: Kolmogorov-Arnold Networks. *arXiv:2404.19756*.
 - [10] Liu, Z., Ma, P., Wang, Y., Matusik, W., and Tegmark, M. (2024b). KAN 2.0: Kolmogorov-Arnold Networks Meet Science. *arXiv:2408.10205*.
 - [11] Moradi, M., Panahi, S., Bollt, E., and Lai, Y.-C. (2024). Kolmogorov-Arnold Network Autoencoders. *arXiv:2410.02077*.
 - [12] Schölkopf, B., Smola, A., and Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319.
 - [13] Wang, T. and Singh, S. (2024). KAN based Autoencoders for Factor Models. *arXiv:2408.02694*.
 - [14] Yu, F., Hu, R., Lin, Y., Ma, Y., Huang, Z., and Li, W. (2024). KAE: Kolmogorov-Arnold Auto-Encoder for Representation Learning. *arXiv:2501.00420*.